# Examen parcial - Fundamentos de Computadores - Turno 1

| | |
|---|---|
| Duración | 1 h 45 min |
| Puntuación total | 10,00 puntos |
| Material permitido | Tabla ASCII / ISO-8859-15 entregada con el examen |
| Instrucciones | Justifica el procedimiento en los ejercicios numéricos. Si no es representable, indícalo. |

## Pregunta 1. (0,50 puntos)

Explica brevemente qué es un computador y el concepto de programa almacenado. Diferencia entre **hardware**, **software**, **bit** y **byte**. ¿Qué características tiene la quinta generación de computadores?

## Pregunta 2. (1,00 puntos)

Responde de forma estructurada:

a) ¿En qué consiste la arquitectura Von Neumann?

b) ¿Qué es una unidad funcional?¿De qué unidades funcionales básicas se compone un computador?

c) Describe el flujo que sigue una instrucción de un programa que necesita leer un dato de memoria principal hasta que se ejecuta en la CPU y el resultado vuelve a quedar disponible. Diferencia que ocurre si se usan enteros o reales.

## Pregunta 3. (1,00 puntos)

Realiza las siguientes conversiones:

a) Convierte $\mathbf{45{,}6875_{10}}$ a **binario**, **octal** y **hexadecimal**.

b) Convierte $\mathbf{2F{,}AD_{16}}$ a **base 10**.

c) Convierte $\mathbf{5072{,}25_{10}}$ a **BCD**.

## Pregunta 4. (1,00 puntos)

a) Usando la tabla suministrada, escribe en **hexadecimal (0x)** la secuencia de bytes necesaria para imprimir en **Windows** la cadena **"Hola 5!"** seguida de **BEL**.

Recuerda que en Windows el salto de línea se representa con **CR + LF**.

b) Se recibe la siguiente tira de bits: **01010011010011110101001100100001**. Di qué texto se transmite.

## Pregunta 5. (2,00 puntos)

a) Completa la siguiente tabla de representación con **6 bits**. Cuando un valor no sea representable, escribe **NR**.

| Decimal | Signo y Magnitud | Complemento a 2 | Exceso Z |
|---|---|---|---|
| +14 | | | |
| -14 | | | |
| +31 | | | |
| -31 | | | |
| +32 | | | |
| -32 | | | |

b) En **Exceso Z** con **5 bits**, dados **A = $10110_Z$** y **B = $01101_Z$**, calcula **S = A + B** y **R = A - B**. Expresa ambos resultados finales en **Exceso Z** y en **base 10**.

c) En **Complemento a 2** y con **6 bits**, calcula **M + N** siendo **M = 011001** y **N = 111011**.

d) En **Complemento a 2** y con **6 bits**, calcula **P - Q** siendo **P = 011110** y **Q = 111011**.

## Pregunta 6.(1,25 puntos)

a) Representa el número decimal **+58,4375** en formato IEEE 754 de **doble precisión (double)**. Indica los campos y la palabra final en binario y hexadecimal.

b) En **de simple precisión (float)**, ¿qué valor representa la palabra **0xC248C000**?

c) Indica el valor del menor número positivo desnormalizado representable en **float**.

d) Escribe una posible representación en **float** de **+∞**, **-∞** y de un **NaN**.

e) Representa en **float** el número decimal $+1{,}2\times10^{40}$

## Pregunta 7. (1,00 puntos)

**7.1. (0,60 puntos)**

Utilizando **solo puertas NAND de 2 entradas**, implementa la función **H(a,b,c)=a·b·c**.

Debes:

- justificar algebraicamente una transformación válida de la función,

- dibujar el circuito equivalente,

- indicar el **coste espacial** (número de puertas) y el **coste temporal** o latencia máxima si cada puerta tarda **1 ns**.

**7.2. (0,40 puntos)**

Dada la función **F(d,c,b,a)=(¬d+¬c)·(a+b)**, obtén la misma función usando **únicamente puertas NAND de 2 entradas**.

Se pide:

- escribir las ecuaciones intermedias,

- dibujar el circuito final.

## Pregunta 8. (1,50 puntos)

### 8.1. Completar una función con enteros (0,70 puntos)

Versión en Python:

```
def polinomio(x, y, z):
   return (3*x - y)/2 + z
```

Completa la subrutina siguiente:

```
.data 0x10010000
x: .word 8
y: .word 5
z: .word 4
res: .space 4
.text 0x00400000
.globl main
main:
    la   a0, x
    la   a1, y
    la   a2, z
    jal  ra, pol_ref
    la   t0, res
    sw   a0, 0(t0)
    li   a7, 10
    ecall
pol_ref:
    # RESOLVER: devolver en a0 = (3*x - y)/2 + z
    ret
```

### 8.2. Programa completo con reales (0,80 puntos)

Versión en Python:

```
pi = 3.1416
r = 2.0
base = 4.0
altura = 3.0
total = (pi*r*r) + (base*altura/2.0)
print(total)
```

Escribe el programa completo, sin subrutinas, a partir del siguiente segmento de datos:

```
.data 0x10010000
pi:     .float 3.1416
r:      .float 2.0
base:   .float 4.0
altura: .float 3.0
dos:    .word 2
total:  .space 4
.text 0x00400000
.globl main
main:
   # ESCRIBE EL PROGRAMA COMPLETO
  Al final finaliza con un print y exit
```

## Pregunta 9. (0,75 puntos)

Partiendo de una dirección inicial alineada a palabra, y sin indicar direcciones concretas, completa la organización de memoria para las siguientes directivas. Indica las zonas no usadas mediante **X**.

```
.data 0x10010000
.byte 9
.half -5, 12
.word 2024
.float 2.5
.ascii "EPSA"
.word -2222
.space 1
.byte 7
```

Rellena la tabla siguiente en orden secuencial de palabras de 32 bits:

| 31..24 | 23..16 | 15..8 | 7..0 | Dirección Memoria |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |

a) ¿Qué cambiaría si se usara **.asciz "EPSA"** en lugar de **.ascii**?

b) ¿Qué cambiaría si se usara **.double 2.5** en lugar de **.float 2.5**? Explica qué bytes de alineación extra aparecerían.